



TSwap Audit Report

Version 1.0

Joev

June 14, 2026

TSwap Audit Report

Joev

June 14, 2026

Prepared by: Joev

Lead Auditors: Jonathan Evan

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
 - High
 - * [H-1] Incorrect fee calculation in `TSwapPool::getInputAmountBasedOnOutput` causes protocol to take too many tokens from users, resulting in lost fees
 - * [H-2] Lack of slippage protection in `TSwapPool::swapExactOutput` causes users to potentially receive way fewer tokens
 - * [H-3] `TSwapPool::sellPoolTokens` mismatches input and output tokens causing users to receive the incorrect amount of tokens

- * [H-4] In `TSwapPool::_swap` the extra tokens given to users after every `swapCount` breaks the protocol invariant of $x * y = k$
- Medium
 - * [M-1] `TSwapPool::deposit` is missing deadline check causing transactions to complete even after the deadline
- Low
 - * [L-1] `TSwapPool::LiquidityAdded` event has parameters out of order
 - * [L-2] Default value returned by `TSwapPool::swapExactInput` results in incorrect return value given
- Informationals
 - * [I-1] `PoolFactory::PoolFactory__PoolDoesNotExist` is not used and should be removed
 - * [I-2] Lacking zero address check in `PoolFactory.sol` constructor
 - * [I-3] `PoolFactory::createPool` should use `.symbol()` instead of `.name()` at `PoolFactory::liquidityTokenSymbol` variable
 - * [I-4] `TSwapPool::Swap` event should be have 3 indexed because there are more than 3 parameters
 - * [I-5] Lacking zero address check in `TSwapPool` constructor

Protocol Summary

This project is meant to be a permissionless way for users to swap assets between each other at a fair price. You can think of T-Swap as a decentralized asset/token exchange (DEX). T-Swap is known as an Automated Market Maker (AMM) because it doesn't use a normal "order book" style exchange, instead it uses "Pools" of an asset. It is similar to Uniswap. To understand Uniswap, please watch this video: [Uniswap Explained](#)

Disclaimer

Joey makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

The findings described in this document correspond the following commit hash:

```
1 e643a8d4c2c802490976b538dd009b351b1c8dda
```

Scope

```
1 ./src/  
2 #-- PoolFactory.sol  
3 #-- TSwapPool.sol
```

Roles

- Liquidity Providers: Users who have liquidity deposited into the pools. Their shares are represented by the LP ERC20 tokens. They gain a 0.3% fee every time a swap is made.
- Users: Users who want to swap tokens.

Executive Summary

The audit process took a week due to the many new vulnerabilities and hacking patterns I hadn't encountered before. I discovered over 12 vulnerabilities for this protocol. In this protocol, I saw and learned a lot vulnerabilities as Weird ERC-20, used Fuzzing test, and others.

Issues found

Severity	Number of issues found
High	4
Medium	1
Low	2
Info	5
Total	12

Findings

High

[H-1] Incorrect fee calculation in TSwapPool::getInputAmountBasedOnOutput causes protocol to take too many tokens from users, resulting in lost fees

Description: The `getInputAmountBasedOnOutput` function is intended to calculate the amount of tokens a user should deposit given an amount of tokens of output tokens. However, the function currently miscalculates the resulting amount. When calculating the fee, it scales the amount by 10_000 instead of 1_000.

Impact: Protocol takes more fees than expected from users.

Recommended Mitigation:

You should change the code to something like this:

```
1     function getInputAmountBasedOnOutput(  
2         uint256 outputAmount,  
3         uint256 inputReserves,  
4         uint256 outputReserves  
5     )  
6     public  
7     pure  
8     revertIfZero(outputAmount)  
9     revertIfZero(outputReserves)  
10    returns (uint256 inputAmount)  
11    {  
12        return  
13 -        ((inputReserves * outputAmount) * 10000) / ((outputReserves  
-        - outputAmount) * 997);
```

```
14 +      ((inputReserves * outputAmount) * 1000) / ((outputReserves
15 -      - outputAmount) * 997);
    }
```

[H-2] Lack of slippage protection in TSwapPool::swapExactOutput causes users to potentially receive way fewer tokens

Description: The `SwapExactOutput` function does not include any sort of slippage protection. This function is similar to what is done in `TSwapPool::swapExactInput`, where the function specifies a `minOutputAmount`, the `swapExactOutput` function should specify a `maxInputAmount`.

Impact: If market conditions change before the transaction processes, the user could get a much worse swap.

Proof of Concept:

You should test with `test_swapExactOutput_hasNoMaxInputAmount_userOverpays()` function in `TSwapPool.t.sol`:

additional `whale` accounts in `setUp()` to help testing:

```
1 function setUp() public {
2   .
3   .
4   .
5 +   weth.mint(whale, 100e18);
6 +   poolToken.mint(whale, 100e18);
7 }
```

After that, you can run the main test:

Proof of Code

```
1 function test_swapExactOutput_hasNoMaxInputAmount_userOverpays()
   public {
2     // PURPOSE: prove that swapExactOutput has NO slippage
       protection.
3     // It has no `maxInputAmount` parameter, so a user cannot cap
       how much
4     // input they are willing to pay. If the price moves against
       them before
5     // their tx executes (volatility / MEV front-run), they
       silently overpay
6     // and the swap STILL succeeds instead of reverting.
7
8     uint256 OUTPUT_AMOUNT = 5e18;    // user wants exactly 5
       poolToken out
```

```
9      uint256 INPUT_RESERVES = 100e18; // calm-market WETH reserve (
      after deposit)
10     uint256 OUTPUT_RESERVES = 100e18; // calm-market poolToken
      reserve (after deposit)
11
12     // Give the user enough WETH to cover the inflated price. setUp
      () already
13     // mints 10e18; the volatile price costs ~222 WETH, so mint 220
      e18 more
14     // (220 + 10 = 230e18). This guarantees the test fails because
      of the BUG,
15     // not because of an insufficient balance.
16     weth.mint(user, 220e18);
17
18     // --- ARRANGE: LP seeds the pool with 100 WETH : 100 poolToken
      ---
19     vm.startPrank(liquidityProvider);
20     weth.approve(address(pool), INPUT_RESERVES);
21     poolToken.approve(address(pool), OUTPUT_RESERVES);
22     pool.deposit(INPUT_RESERVES, 90e18, OUTPUT_RESERVES, uint64(
      block.timestamp));
23     vm.stopPrank();
24
25     // Baseline "fair price": the input the user SHOULD pay for
      OUTPUT_AMOUNT
26     // in the calm 100:100 pool. Captured BEFORE the whale moves
      the price.
27     // NOTE: this value is itself ~10x too high due to a SEPARATE
      bug in
28     // getInputAmountBasedOnOutput (magic number 10000 should be
      1000), but
29     // it still works as a relative baseline for this slippage
      proof.
30     uint256 expectedNormal = pool.getInputAmountBasedOnOutput(
      OUTPUT_AMOUNT, INPUT_RESERVES, OUTPUT_RESERVES);
31
32     // --- ACT 1: the whale creates volatility ---
33     // A large swap drains poolToken, making it scarce/expensive.
      This stands
34     // in for a front-run / market move landing just before the
      user's tx.
35     vm.startPrank(whale);
36     weth.approve(address(pool), 100e18);
37     pool.swapExactInput(weth, 100e18, poolToken, 40e18, uint64(
      block.timestamp));
38     vm.stopPrank();
39
40     // --- ACT 2: the victim swaps at the now-worse price ---
41     vm.startPrank(user);
42     uint256 startingUserBalance = weth.balanceOf(user);
43     weth.approve(address(pool), type(uint256).max);
```

```
44     pool.swapExactOutput(weth, poolToken, OUTPUT_AMOUNT, uint64(
45         block.timestamp));
46     uint256 endingUserBalance = weth.balanceOf(user);
47     vm.stopPrank();
48     // Actual WETH that left the user's wallet for the same 5
49     // poolToken.
50     uint256 wethUserPaid = startingUserBalance - endingUserBalance;
51     // --- ASSERT: the user paid far more than the fair price, yet
52     // the swap
53     // succeeded. With a maxInputAmount this call would have
54     // reverted. ---
55     assert(wethUserPaid > expectedNormal * 2);
56     console.log("Expected WETH User Paid (calm price): ",
57         expectedNormal);
58     // 52.789948793749670062 WETH
59     console.log("Actual WETH User Paid (after volatility): ",
60         wethUserPaid);
61     // 222.519471985918317079 WETH -> ~4.2x the calm price, with
62     // NO revert
63 }
```

Recommended Mitigation:

We should include a `maxInputAmount` so the user only has to spend up to a specific amount and can predict how much they will spend on the protocol.

```
1     function swapExactOutput(
2         IERC20 inputToken,
3         uint256 maxInputAmount,
4         .
5         .
6         .
7         inputAmount = getInputAmountBasedOnOutput(
8             outputAmount,
9             inputReserves,
10            outputReserves
11        );
12     +     if (inputAmount > maxInputAmount) {
13     +         revert();
14     +     }
15
16     _swap(inputToken, inputAmount, outputToken, outputAmount);
```

[H-3] TSwapPool::sellPoolTokens mismatches input and output tokens causing users to receive the incorrect amount of tokens

Description: The `sellPoolTokens` function is intended to allow users to easily sell pool tokens and receive WETH in exchange. Users indicate how many pool tokens they're willing to sell in the `poolTokenAmount` parameter. However, the function currently miscalculates the swapped amount.

This is due to the fact that the `swapExactOutput` function is called, whereas the `swapExactInput` function is the one that should be called. Because users specify the exact amount of input tokens, not output.

Impact: User will swap the wrong amount of tokens, which is a severe disruption of protocol functionality.

Proof of Concept:

You should test with `test_sellPoolTokens_swapsInputAndOutput()` function in `TSwapPool.t.sol`:

Proof of Code

```
1      function test_sellPoolTokens_swapsInputAndOutput() public {
2          uint256 INPUT_RESERVES = 100e18;
3          uint256 OUTPUT_RESERVES = 100e18;
4
5          // Extra mint: because of the bug the user is charged FAR more
6          // pool tokens than
7          // intended (~20e18). Without this the tx would revert on
8          // insufficient balance,
9          // which is itself a secondary impact of the bug.
10         poolToken.mint(user, 20e18);
11
12         // Seed the pool with liquidity so the swap has reserves to
13         // price against.
14         vm.startPrank(LiquidityProvider);
15         weth.approve(address(pool), INPUT_RESERVES);
16         poolToken.approve(address(pool), OUTPUT_RESERVES);
17         pool.deposit(INPUT_RESERVES, 90e18, OUTPUT_RESERVES, uint64(
18             block.timestamp));
19         vm.stopPrank();
20
21         // The user's intent: sell exactly 2 pool tokens.
22         uint256 expectedPoolTokenSold = 2e18;
```

```
uint256 startingUserPoolTokenBalance = poolToken.balanceOf(user);
uint256 startingUserWethBalance = weth.balanceOf(user);
```

```
23     vm.startPrank(user);
24     poolToken.approve(address(pool), type(uint256).max);
25     pool.sellPoolTokens(expectedPoolTokenSold);
26     vm.stopPrank();
27
28     uint256 endingUserPoolTokenBalance = poolToken.balanceOf(user);
29     uint256 endingUserWethBalance = weth.balanceOf(user);
30
31     // SYMPTOM: the user actually pays more pool tokens than they
32     // meant to sell.
33     uint256 actualPoolTokenSold = startingUserPoolTokenBalance -
34     endingUserPoolTokenBalance;
35     assertGt(actualPoolTokenSold, expectedPoolTokenSold);
36
37     console.log("Actual Pool Token sold: ", actualPoolTokenSold);
38     // 20.469571981249872065 poolToken
39     console.log("Token Pool user wants to sold: ",
40     expectedPoolTokenSold);
41     // 2.000000000000000000 poolToken
42
43     // ROOT CAUSE: the user receives exactly `expectedPoolTokenSold
44     // ` worth of WETH,
45     // proving the value was used as the OUTPUT (WETH out) instead
46     // of the INPUT
47     // (pool tokens to sell).
48     uint256 userWethReceive = endingUserWethBalance -
49     startingUserWethBalance;
50
51     assert(userWethReceive == expectedPoolTokenSold);
52     console.log("User Weth Receive: ", userWethReceive);
53     // 2.000000000000000000 WETH
54     console.log("Token Pool user wants to sold: ",
55     expectedPoolTokenSold);
56     // 2.000000000000000000 poolToken
57 }
```

Recommended Mitigation:

consider changing the implementation to use `swapExactInput` instead of `swapExactOutput`. Note that this would also require changing the `sellPoolToken` function to accept a new parameter (ie `minWethToReceive` to be passed to `swapExactInput`).

```
1     function sellPoolTokens(
2         uint256 poolTokenAmount,
3     +     uint256 minWethToReceive,
4     ) external returns (uint256 wethAmount) {
5         return
6     -     swapExactOutput(
7     -         i_poolToken,
8     -         i_wethToken,
9     -         poolTokenAmount,
```

```

10 -         uint64(block.timestamp)
11 -     );
12 +     swapExactInput(
13 +         i_poolToken,
14 +         poolTokenAmount,
15 +         i_wethToken,
16 +         minWethToReceive
17 +         uint64(block.timestamp)
18 +     );
19 }

```

Additionally, it might be wise to add a deadline to the function, as there is currently no deadline.

[H-4] In TSwapPool : : _swap the extra tokens given to users after every swapCount breaks the protocol invariant of $x * y = k$

Description: The protocol follow a strict invariant of $x * y = k$. Where: - x : The balance of the pool token - y : The balance of WETH - k : the constant product of the two balances

This means, that whenever the balances change in the protocol, the ratio between the two amount should remain constant, hence the k . However, this is broken due to the extra incentive in the `_swap` function. Meaning that over time the protocol funds will be drained.

The follow block of code is responsible for the issue.

```

1     swap_count++;
2     if (swap_count >= SWAP_COUNT_MAX) {
3         swap_count = 0;
4         outputToken.safeTransfer(msg.sender, 1
5             _000_000_000_000_000_000);
6     }

```

Impact: A user could maliciously drain the protocol of funds by doing a lot of swaps and collecting the extra incentive given out by the protocol.

Most simply put, the protocol's core invariant is broken.

Proof of Concept: 1. A user swaps 10 times and collects the extra incentive of 1_000_000_000_000_000_000 tokens 2. That user continues the swap until all the protocol funds are drained

Proof of Code

Place the following into `TSwapPool.t.sol`.

```

1     function testInvariantBroken() public {
2         vm.startPrank(liquidityProvider);
3         weth.approve(address(pool), 100e18);
4         poolToken.approve(address(pool), 100e18);

```

```
5      pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
6      vm.stopPrank();
7
8      uint256 outputWeth = 1e17;
9
10     vm.startPrank(user);
11     poolToken.approve(address(pool), type(uint256).max);
12     poolToken.mint(user, 100e18);
13     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.
14         timestamp));
15     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.
16         timestamp));
17     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.
18         timestamp));
19     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.
20         timestamp));
21     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.
22         timestamp));
23     int256 startingY = int256(weth.balanceOf(address(pool)));
24     int256 expectedDeltaY = int256(-1) * int256(outputWeth);
25
26     pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.
27         timestamp));
28     vm.stopPrank();
29
30     uint256 endingY = weth.balanceOf(address(pool));
31     int256 actualDeltaY = int256(endingY) - int256(startingY);
32     assertEq(actualDeltaY, expectedDeltaY);
33 }
```

Recommended Mitigation: Remove the extra incentive mechanism. If you want to keep this in, you should account for the change in the $x * y = k$ protocol invariant. Or, you should set aside tokens in the same way you do with fees.

```
1 -     swap_count++;
2 -     if (swap_count >= SWAP_COUNT_MAX) {
3 -         swap_count = 0;
4 -         outputToken.safeTransfer(msg.sender, 1
5 -             _000_000_000_000_000_000);
6 -     }
```

Medium

[M-1] TSwapPool::deposit is missing deadline check causing transactions to complete even after the deadline

Description: The `deposit` function accepts a deadline parameter, which according to the documentation is “The deadline for the transaction to be completed by”. However, this parameter is never used. As a consequence, operations that add liquidity to the pool might be executed at unexpected times, in market conditions where the deposit rate is unfavorable.

Impact: Transactions could be sent when market conditions are unfavorable to deposit, even when adding a deadline parameter.

Proof of Concept: The `deadline` parameter is unused.

Contests of the compiler:

```
1  forge build
2
3  Warning (5667): Unused function parameter. Remove or comment out the
   variable name to silence this warning.
4      --> src/TSwapPool.sol:124:9:
5          |
6  124  |           uint64 deadline
7          |           ^^^^^^^^^^^^^^^^^
```

Recommended Mitigation: Consider making the following change to the function.

```
1      function deposit(
2          uint256 wethToDeposit,
3          uint256 minimumLiquidityTokensToMint,
4          uint256 maximumPoolTokensToDeposit,
5          uint64 deadline
6      )
7      {
8          external
9          + revertIfDeadlinePassed(deadline)
10             revertIfZero(wethToDeposit)
11             returns (uint256 liquidityTokensToMint)
12     }
```

Low

[L-1] TSwapPool::LiquidityAdded event has parameters out of order

Description: When the `LiquidityAdded` event is emitted in the `_addLiquidityMintAndTransfer` function, it logs values in an incorrect order. The `poolTokenToDeposit` value should go in the

third parameter position, whereas the `wethToDeposit` value should go second.

Impact: Event emission is incorrect, leading to off-chain function potentially malfunctioning.

Recommended Mitigation:

```
1 -   emit LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit)
    ;
2 +   emit LiquidityAdded(msg.sender, wethToDeposit, poolTokensToDeposit)
    ;
```

[L-2] Default value returned by TSwapPool : : swapExactInput results in incorrect return value given

Description: The `swapExactInput` function is expected to return the actual amount of token bought by the caller. However, while it declared the name return value `output`, it is never assigned a value, nor uses an explicit return statement.

Impact: The return value will always be 0, giving incorrect information to the caller.

Proof of Concept:

You should test with `test_swapExactInput_returnsZeroOutput()` function in `TSwapPool.t.sol`:

Proof of Code

```
1   function test_swapExactInput_returnsZeroOutput() public {
2       vm.startPrank(liquidityProvider);
3       weth.approve(address(pool), 100e18);
4       poolToken.approve(address(pool), 100e18);
5       pool.deposit(100e18, 90e18, 100e18, uint64(block.timestamp));
6       vm.stopPrank();
7
8       vm.startPrank(user);
9       uint256 startingPoolTokenBalanceUser = poolToken.balanceOf(user
10      );
11      weth.approve(address(pool), 5e18);
12      uint256 output = pool.swapExactInput(weth, 5e18, poolToken, 3
13      e18, uint64(block.timestamp));
14
15      uint256 endingPoolTokenBalanceUser = poolToken.balanceOf(user);
16      console.log("Starting Pool Token Balance User: ",
17      startingPoolTokenBalanceUser);
18      console.log("Ending Pool Token Balance User: ",
19      endingPoolTokenBalanceUser);
20
21      assert(output == 0);
```

```
18         assert(endingPoolTokenBalanceUser >
19                startingPoolTokenBalanceUser);
20     vm.stopPrank();
21 }
```

Recommended Mitigation:

You should change the code to something like this:

```
1     {
2         uint256 inputReserves = inputToken.balanceOf(address(this));
3         uint256 outputReserves = outputToken.balanceOf(address(this));
4
5 -         uint256 outputAmount = getOutputAmountBasedOnInput(
6 +         output = getOutputAmountBasedOnInput(
7             inputAmount,
8             inputReserves,
9             outputReserves
10        );
11
12 -         if (outputAmount < minOutputAmount) {
13 -             revert TSwapPool__OutputTooLow(outputAmount,
14         minOutputAmount);
15 +         }
16 +         if (output < minOutputAmount) {
17 +             revert TSwapPool__OutputTooLow(output, minOutputAmount);
18         }
19
20 -         _swap(inputToken, inputAmount, outputToken, outputAmount);
21 +         _swap(inputToken, inputAmount, outputToken, output);
22     }
```

Informationals

[I-1] PoolFactory::PoolFactory__PoolDoesNotExist is not used and should be removed

```
1 - error PoolFactory__PoolDoesNotExist(address tokenAddress);
```

[I-2] Lacking zero address check in PoolFactory.sol constructor

```
1     constructor(address wethToken) {
2 +         if (wethToken == address(0)) {
3 +             revert();
4 +         }
```

```
5         i_wethToken = wethToken;
6     }
```

[I-3] PoolFactory::createPool should use .symbol() instead of .name() at PoolFactory::liquidityTokenSymbol variable

```
1 -     string memory liquidityTokenSymbol = string.concat("ts", IERC20(
    tokenAddress).name());
2 +     string memory liquidityTokenSymbol = string.concat("ts", IERC20(
    tokenAddress).symbol());
```

[I-4] TSwapPool::Swap event should be have 3 indexed because there are more than 3 parameters

`tokenIn` dan `tokenOut` should be indexed to enable efficient off-chain filtering by token address. Amounts do not benefit from indexing due to Solidity's inability to perform range queries on topics.

```
1     event Swap(
2         address indexed swapper,
3 -         IERC20 tokenIn,
4 +         IERC20 indexed tokenIn,
5         uint256 amountTokenIn,
6 -         IERC20 tokenOut,
7 +         IERC20 indexed tokenOut,
8         uint256 amountTokenOut
9     );
```

[I-5] Lacking zero address check in TSwapPool constructor

```
1     constructor(
2         address poolToken,
3         address wethToken,
4         string memory liquidityTokenName,
5         string memory liquidityTokenSymbol
6     ) ERC20(liquidityTokenName, liquidityTokenSymbol) {
7 +         if (wethToken == address(0)) {
8 +             revert();
9 +         }
10 +         if (poolToken == address(0)) {
11 +             revert();
12 +         }
13         i_wethToken = IERC20(wethToken);
14         i_poolToken = IERC20(poolToken);
15     }
```